Lab 17 — Helm and delivery

Theory — how a stack of eleven manifests becomes a versioned package that one command installs, upgrades and rolls back; how an image travels from CI to the cluster; and what changes when the cluster is no longer your own k3s.

Objectives

- Read a **Helm chart**: `Chart.yaml`, `values.yaml`, `templates/`; understand *release* and *revision*.
- Install, upgrade, roll back: `helm install / upgrade / rollback / history / uninstall`.
- Do the same without templates with **Kustomize** (base + overlays), and know which to pick.
- Adopt a **tag and digest strategy**, and make k3s pull from a registry.
- Place the **CI/CD → cluster** chain, **GitOps**, and k3s against EKS/AKS/GKE/OpenShift.

---

1. From loose YAML to a package

In lab 16 the stack was a dozen manifests: Deployments, Services, ConfigMaps, Secret, StatefulSet, Ingress, LimitRange. To deliver them to staging and then production, you would copy them and change the replica count, the image tag, the Spring profile, the Ingress host… in duplicated files. And when version 1.1 breaks, dig by hand for what was there before. It is exactly the problem `compose.override.yaml` solved in lab 09 for one machine. Kubernetes has two complementary answers: **Helm** (templates and values) and **Kustomize** (patches over plain YAML).

> **→ DOCKER RECAP**
> `compose.yaml` + `compose.prod.yaml` ↔ `values.yaml` + `values-prod.yaml` ; `podman compose up` ↔ `helm install` ; `podman compose down` ↔ `helm uninstall` (which, like `down` without `-v`, keeps the volumes); `podman kube play` (lab 11) also installed a YAML, but with no history and no values. The tags and digests of lab 10 are unchanged: same images, same `localhost:5000` registry.

2. Helm: chart, release, revision

A **chart** is a directory (or a `.tgz` archive):

```
stack/
├── Chart.yaml          # name, chart version, appVersion
├── values.yaml         # every default value — what varies per environment
├── values-prod.yaml    # overrides for one environment (only the differences)
├── templates/          # the manifests, with holes: {{ .Values.api.replicas }}
│   ├── api.yaml  web.yaml  db.yaml  configmap.yaml  secret.yaml  ingress.yaml
│   ├── _helpers.tpl    # reusable snippets (common labels)
│   └── NOTES.txt       # text printed after installation
└── files/              # files injected verbatim: {{ .Files.Get "files/nginx-proxy.conf" }}
```

Templates are **Go templates**: `{{ .Values.api.image.tag }}`, `{{ .Release.Name }}`, `{{ if .Values.web.ingress.enabled }}`, `{{ toYaml .Values.api.resources | nindent 12 }}`. `helm template` renders the YAML without installing anything; `helm lint` checks the chart. A **release** is a chart installed under a name in a namespace; every `install`, `upgrade` or `rollback` creates a

revision, which Helm stores in a Secret `sh.helm.release.v1.<name>.v<n>` of the namespace: that is what makes `helm history` and `helm rollback` possible without a server.

Command	Effect
`helm install stack ./stack -n lab17 --create-namespace`	revision 1; prints `NOTES.txt`
`helm upgrade stack ./stack --set api.replicas=3 -f values-prod.yaml`	next revision; `--reuse-values` keeps earlier `--set`s
`helm rollback stack 1`	new revision "Rollback to 1" (you do not go back, you go forward)
`helm history` / `helm get values` / `helm get manifest`	what is installed, with which values
`helm uninstall stack`	removes the chart's objects — **not** the PVCs from `volumeClaimTemplates`

One detail matters: changing a ConfigMap does not restart Pods (lab 14). The chart annotates the Pod template with `checksum/config: {{ … | sha256sum }}`; any change to the ConfigMap changes the annotation, hence triggers a rolling update. It is the most copied Helm idiom.

> **→ HISTORY**
> Helm was born in 2015 at Deis as "apt for Kubernetes", joined the CNCF and became the standard for public *charts* (databases, Prometheus, ingress controllers…). Helm 2 required a server component, Tiller, too powerful for security teams' taste; Helm 3 (2019) removed it: the client talks directly to the API server with your kubeconfig and your RBAC rights. Helm 4 (late 2025) keeps the same CLI. Kustomize, born at Google in 2017, has been built into `kubectl` since 2019 (`kubectl apply -k`).

> **K3S**
> You have been using Helm since lab 11 without knowing it: k3s installs Traefik through a `HelmChart` object (a CRD) that its `klipper-helm` controller applies; `helm list -A` shows the `traefik` and `traefik-crd` releases in `kube-system`. The `helm` binary is not shipped: you download it, and it does not know the k3s kubeconfig: without `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` it looks for a cluster on `localhost:8080` and fails.

3. Kustomize: patches, not templates

Kustomize starts from a **base** (valid YAML, applicable as is) and per-environment **overlays** that reference and modify it: `namespace`, `replicas`, `images` (change a tag without touching the YAML), `labels`, and `patches` (strategic merge or JSON patch). No `{{ }}`, no values: every overlay is readable on its own, and `kubectl kustomize overlays/prod` shows the exact result; `kubectl apply -k overlays/prod` applies it. Many teams combine both: `helm template … | kustomize` to patch a public chart they do not want to fork.

	Helm	Kustomize
Model	templates + values	plain YAML + patches
Distribution	versioned charts, repositories, `.tgz`	a Git repository
History / rollback	yes (revisions)	no (Git carries it)
Suited to	software installed from others (public charts, internal products)	your own apps, especially with GitOps

4. Tags, digests and registry

A **tag** is mutable: `api:1.1` may point to another image tomorrow, and `latest` is a promise of surprises. Two company practices: **immutable** tags (the Git SHA or a SemVer that is never rewritten, set by CI), and, for what runs in production, the **digest**: `image: localhost:5000/api@sha256:b82020cb…` names exactly one manifest, forever. `podman push --digestfile` gives the digest as the registry recorded it — different from `podman image inspect --format '{{.Digest}}'` on the local image, because the push recompresses the layers; and `kubectl get pod -o jsonpath='{.status.containerStatuses[0].imageID}'` shows the one the node actually pulled.

So far, images went through `podman save | sudo k3s ctr images import`. The real path is a **registry**: CI pushes, the cluster pulls. The `registry:2` of lab 10 is enough: k3s pulls `localhost:5000/api:1.1` over HTTP with no configuration (containerd tolerates plain `localhost`). For a company registry (internal TLS, credentials, Docker Hub mirror), k3s reads `/etc/rancher/k3s/registries.yaml` (`mirrors:` and `configs:` with `auth:` / `tls:`) and needs `sudo systemctl restart k3s`. On any other cluster, credentials go into a `kubernetes.io/dockerconfigjson` Secret referenced by `imagePullSecrets`.

> **→ SECURITY**
> A mutable tag is a hole in reproducibility and an entry point (who is allowed to push `api:1.1`?). The digest closes the door; `imagePullPolicy: Always` with a mutable tag only moves it. Serious pipelines sign images (cosign) and reject at admission (Kyverno, Gatekeeper) whatever is unsigned or unscanned. And the registry password is never in the chart: `--set` from the CI vault, or an external Secret (lab 14).

5. CI/CD → cluster, and GitOps

The lab 10 chain gains one more stage:

```
deploy:                                  # pipeline excerpt (GitLab CI, GitHub Actions: same idea)
  script:
    - podman build -t registry.internal/api:$CI_COMMIT_SHA .
    - podman push registry.internal/api:$CI_COMMIT_SHA
    - helm upgrade --install api ./chart -n billing-prod -f values-prod.yaml
        --set api.image.tag=$CI_COMMIT_SHA --wait --timeout 5m
```

The *runner* holds a kubeconfig limited to a ServiceAccount (lab 16) with `edit` on **one** namespace; `--wait` fails the job if the rollout does not converge, and a `helm rollback` is the pipeline's next step on failure. **GitOps** reverses the direction: nobody runs `helm upgrade`; a Git repository

describes the desired state (charts + values, or Kustomize overlays), and an agent inside the cluster (**Argo CD**, **Flux**) reads it continuously and reconciles — the lab 11 loop, one floor up. CI only builds, pushes, and **changes a tag in Git**; history, review and rollback are Git's.

6. k3s versus company clusters

	k3s (what you have)	**EKS / AKS / GKE**	**OpenShift**
Control plane	one binary, on your machine	managed by the cloud, billed	managed (OpenShift) or installed
`Service: LoadBalancer`	ServiceLB (node ports)	cloud load balancer, public IP	same + **Route** (its own Ingress)
StorageClass	`local-path` (node disk)	network disks (EBS, Azure Disk, PD)	same
Ingress	Traefik preinstalled	to install (nginx, ALB…)	built-in Router
Identity	kubeconfig = `cluster-admin`	IAM / Entra / Google → RBAC	built-in OAuth, SCC (strict security, no root)
CLI	`kubectl`	`kubectl`	`oc` (superset)

Everything else — Pod, Deployment, Service, ConfigMap, PVC, RBAC, Helm — is **identical**: that is the point of a standard API. What changes sits at the edge: who provides the load balancer, the disk, the identity, and what it costs.

7. In the company

- **One chart per application**, versioned with it; the `values-<env>.yaml` in the same repository, the secrets elsewhere.
- **Immutable tags set by CI**, digests in production, `latest` forbidden.
- **The CI runner has a role, not `cluster-admin`**; better still, GitOps, and CI no longer touches the cluster.
- **Helm for what you install, Kustomize for what you write** — or both, without dogma.
- **k3s is a development cluster** faithful to the API; what you learned on it carries over unchanged.

Remember

- Chart = `Chart.yaml` + `values.yaml` + `templates/`; release = installed chart; revision = every `install/upgrade/rollback`, stored in a Secret.
- `helm upgrade … --set` / `-f values-prod.yaml`, `helm history`, `helm rollback 1` (which creates a revision); `uninstall` keeps PVCs.
- `checksum/config` on the Pod template: a changed ConfigMap redeploys.
- Kustomize = base + overlays (`namespace`, `replicas`, `images`, `patches`), `kubectl apply -k`, no template at all.
- Mutable tag ≠ digest: `--digestfile` at push, `imageID` on the node, `image@sha256:` in production.

- k3s pulls `localhost:5000` with no config; a company registry goes through `registries.yaml` or `imagePullSecrets`.
- CI: build → push → `helm upgrade --install --wait` with a limited role; GitOps: Git describes, Argo CD/Flux reconciles.

Vocabulary

chart: Helm package (templates + values). — **release / revision**: chart installed under a name / numbered version of that installation. — **values**: chart parameters, overridden with `-f` or `--set`. — **`helm template`**: rendering without installation. — **Kustomize**: base + overlays, patches without templates. — **overlay**: per-environment variant. — **digest**: immutable fingerprint of an image (`@sha256:`). — **registries.yaml**: k3s registry configuration. — **imagePullSecrets**: a Pod's registry credentials. — **GitOps**: Git as the source of truth, reconciled by an agent (Argo CD, Flux). — **managed cluster**: control plane provided by a cloud (EKS, AKS, GKE). — **OpenShift**: Red Hat's Kubernetes distribution (Routes, SCC, `oc`).